

Q.No:3

Question description

You are given an $n \times n$ grid representing the map of a forest. Each square is either empty or contains a tree. The upper-left square has coordinates $(1,1)$, and the lower-right square has coordinates (n,n) .

Your task is to process q queries of the form: how many trees are inside a given rectangle in the forest?

Constraints

- $1 \leq n \leq 1000$
- $1 \leq q \leq 10^5$
- $1 \leq y1 \leq y2 \leq n$
- $1 \leq x1 \leq x2 \leq n$

Input

The first input line has two integers n and q : the size of the forest and the number of queries.

Then, there are n lines describing the forest. Each line has n characters: $.$ is an empty square and $*$ is a tree.

Finally, there are q lines describing the queries. Each line has four integers $y1, x1, y2, x2$ corresponding to the corners of a rectangle.

Output

Print the number of trees inside each rectangle.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
4 3
.*..
*.*
*.*
*.*
****
2 2 3 4
3 1 3 1
1 1 2 2
```

EXPECTED OUTPUT

```
3
1
2
```

Test Case 2

INPUT (STDIN)

```
4 3
.*.*
*.*
*.*
*.*
****
1 2 3 4
3 2 3 4
1 4 2 3
```

EXPECTED OUTPUT

```
6
2
0
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
for(i=1;i<=n;i++)
```

Complexity Test Cases

Code:

```
#include <stdio.h>

int main()
{
    int n, q;
    int a[1001][1001] = {0};

    scanf("%d %d", &n, &q);
    for (int i = 1; i <= n; i++)
    {
        char s[1001];

        scanf("%s", s);

        for (int j = 1; j <= n; j++)
        {
            if (s[j - 1] == '*')
```

```
        a[i][j] = 1;

        a[i][j] = a[i][j]
            + a[i - 1][j]
            + a[i][j - 1]
            - a[i - 1][j - 1];
    }
}
int y1[100000], x1[100000];
int y2[100000], x2[100000];

for (int i = 0; i < q; i++)
{
    scanf("%d %d %d %d",
        &y1[i], &x1[i],
        &y2[i], &x2[i]);
}

printf("Output:\n");
for (int i = 0; i < q; i++)
{
    int ans;

    ans = a[y2[i]][x2[i]]
        - a[y1[i] - 1][x2[i]]
        - a[y2[i]][x1[i] - 1]
        + a[y1[i] - 1][x1[i] - 1];

    printf("%d\n", ans);
}

return 0;
}
```

Output:

Test_case 1:

```
4 3
.*..
*.*
**..
****
2 2 3 4
3 1 3 1
1 1 2 2
Output:
3
1
2
```

Test_case 2:

```
aadityajaiswal@Aadityas-MacBook  
4 3  
.*. *  
*.* *  
**.*  
***.  
1 2 3 4  
3 2 3 4  
1 4 2 3  
Output:  
6  
2  
0  
aadityajaiswal@Aadityas-MacBook
```